EX16 - Procesamiento de trayectoria - Mascaras booleanas

Guia paso a paso para principiantes

Esta guia sigue el archivo real `ex16_trajectory_data_processing.py`. Se explica primero el problema y despues el codigo desde el punto de vista de alguien que esta empezando Python.

1. Que queremos resolver

Responder preguntas sobre la trayectoria con condiciones vectorizadas: ascenso, descenso, alturas, ventanas temporales, estadistica, indices, energia y dt.

Variables / elementos del problema

Nombre	Significado
mask	array de True/False
processed_data	matriz [h,v,t,energy]

Ecuacion o logica en papel

```
mask = condition on a column
filtered = A[mask]
dt_k = t[k+1] - t[k]
```

Comprobacion manual

Si velocities=[2,-1,3], velocities>0 produce [True,False,True]. Aplicar esa mascara al dataset conserva las filas 0 y 2.

2. Lo que ya sabemos y lo nuevo

Ya visto:

- arrays
- slicing de columnas
- vectorizacion

Nuevo en este ejercicio:

- mascara booleana
- A[mask]
- &
- np.sum(mask)
- np.where
- mean/std
- column_stack
- diff
- allclose

3. Conceptos Python nuevos

1. Mascara

Sintaxis: `ascending_mask = A[:,1] > 0`

La comparacion se aplica a cada elemento y devuelve True/False por muestra.

2. Filtrado

Sintaxis: `A[mask]`

NumPy conserva las filas cuya posicion correspondiente en mask es True.

3. &

Sintaxis: `(times >= 2) & (times <= 5)`

Con arrays usamos & para exigir ambas condiciones elemento a elemento.

4. np.where

Sintaxis: `np.where(mask)[0]`

Devuelve las posiciones donde la mascara es True.

5. np.diff

Sintaxis: `np.diff(times)`

Calcula diferencias consecutivas, por eso N muestras producen N-1 intervalos.

4. Codigo paso a paso

Paso 1. Crear mascaras

```
ascending_mask = trajectory_array[:, 1] > 0
descending_mask = trajectory_array[:, 1] < 0
```

Primero seleccionamos velocidad y despues comparamos todos sus valores.

Paso 2. Filtrar estados

```
ascending_states = trajectory_array[ascending_mask]
```

Aplicar la mascara a la matriz completa devuelve filas [h,v,t].

Paso 3. Combinar condiciones

```
time_window = ((trajectory_array[:,2] >= 2) &
               (trajectory_array[:,2] <= 5))
```

Cada comparacion genera una mascara; & exige True en ambas.

Paso 4. Estadistica

```
mean_height = np.mean(trajectory_array[:, 0])
std_height = np.std(trajectory_array[:, 0])
```

Las funciones reciben una columna numerica y devuelven escalares.

Paso 5. Anadir energia

```
processed_data = np.column_stack((trajectory_array, energy))
```

energy tiene un valor por fila y se anade como cuarta columna.

Paso 6. Comprobar dt

```
dt_samples_diff = np.diff(trajectory_array[:, 2])
dt_samples_allclose = np.allclose(dt_samples_diff, config["dt"])
```

diff obtiene todos los intervalos y allclose comprueba si son cercanos a dt dentro de tolerancia.

5. Flujo de ejecucion

```
trajectory matrix
 -> masks
 -> filtered rows / statistics / indices
 -> energy column
 -> processed_data
 -> dt check
```

6. Resultado / comprobacion esperada

El chequeo de dt debe indicar Time Matches y processed_data debe tener cuatro columnas.

7. Errores tipicos al empezar

- Usar and en lugar de & con arrays.
- Olvidar parentesis en condiciones compuestas.
- Confundir np.sum(mask) con indices.
- Esperar N valores de np.diff para N muestras.

8. Si solo recuerdas cinco cosas

- Una mascara es un array booleano.
- A[mask] filtra datos.
- & combina condiciones.
- np.where localiza posiciones.
- np.diff y allclose permiten validar el muestreo.

9. Mini practica

Cambia un unico input. Antes de ejecutar, escribe en un comentario que esperas que cambie y por que. Despues ejecuta y compara la prediccion con la salida.

Criterio de cierre

Puedes continuar cuando puedas explicar el problema, la ecuacion/logica, los conceptos Python nuevos y el flujo de ejecucion sin limitarte a leer el codigo.